Classification of Plant Leaf Diseases Using Deep Learning: A Tomato Case Study

Vardhini Talluri

COMP-671 Deep Learning, Fall 2026 · Final Project Report
Code: github.com/Rivier-Computer-Science/FA26-COMP-671-Talluri (branch `Talluri`)

Abstract

Growers often learn that a crop is diseased only after the damage is done, and expert diagnosis is not available to everyone. We ask how well a convolutional network can recognize nine tomato leaf diseases, plus healthy leaves, from a single photo, and which design choices matter. We use the 18,155 tomato images of PlantVillage, split by physical leaf using the dataset's grouping data (37% of images lack it, so some leakage cannot be excluded). Five models are compared: a handcrafted color and texture SVM, a small CNN trained from scratch, and a ResNet-18 that is either randomly initialized, or ImageNet-pretrained with or without data augmentation. Macro-F1 on the held-out test set is our main metric. Pretraining is the largest effect: it raises macro-F1 from 0.940 to 0.980 (paired 95% interval for the gain: +0.028 to +0.053). The SVM baseline reaches 0.959 and beats both models trained from scratch. Augmentation makes no measurable difference on clean test images (0.980 vs. 0.985) but keeps the model usable under blur and darkening, where the unaugmented model falls apart. Its mistakes also lean toward calling diseased leaves healthy (16 of 68 errors, versus none for the unaugmented model). All models degrade sharply under even mild image noise, and the one-crop, laboratory-background data means these scores should not be read as field performance.

Keywords: plant disease classification, convolutional neural networks, ResNet, transfer learning, data augmentation, failure analysis

1. Problem Statement and Motivation

Task. The input is a color photograph of one tomato leaf. The output is one of ten labels: healthy, or one of nine diseases (bacterial spot, early blight, late blight, leaf mold, Septoria leaf spot, spider-mite damage, target spot, yellow leaf curl virus, mosaic virus). It is a single-label, ten-way image classification problem.

Why it matters. Crop diseases cause large yield losses, and the people who lose most are often small growers who cannot easily reach an agronomist. A phone app that flags a likely disease early could help them decide when to treat and when to call for expert advice. That motivation also tells us what to worry about: an app that is confidently wrong can cost a grower a crop, and the original PlantVillage study itself reported accuracy falling from 99.35% on held-out laboratory images to 31.4% on images taken under other conditions [1]. This project therefore asks more than "what is the accuracy?"

Questions. (1) How much do deep models improve on a simple non-deep baseline? (2) How much does ImageNet pretraining matter? (3) Does data augmentation help, both on clean test images and on degraded ones? (4) Where and why does the best model fail?

Success criteria. The primary metric is macro-averaged F1 on a test set whose leaves never appear in training; we also report accuracy and per-class F1. Macro-F1 weights every class equally, which matters here because the rarest class has about 2% of the images. We read a result as meaningful when (a) a model beats the handcrafted baseline by more than test-set sampling noise (the 95% bootstrap interval of the paired difference excludes zero), (b) no class has F1 below 0.90, and (c) we can say specifically where and why the remaining errors occur. These criteria are how we judge the results below; they are not a claim that we fixed thresholds before running anything.

Scope. One crop, ten classes, about 18,000 images, and a laptop-sized compute budget (about an hour of total training time). That is small enough to run every comparison to completion and still examine failures in detail. It is also the reason we make no claim about field deployment.

Course connection. The work draws on the course text [2]: loss functions and fitting (Chapters 5–6), measuring performance and generalization (Chapter 8), regularization, data augmentation and transfer learning (Chapter 9), convolutional networks (Chapter 10), and residual networks with batch normalization (Chapter 11).

2. Data

2.1 Dataset

We use PlantVillage [3, 1], a public collection of leaf photographs taken against a plain background, as redistributed on Hugging Face (`mohanty/PlantVillage`, CC-BY-SA-3.0). We keep the color images of the ten tomato classes: 18,155 images, each showing one leaf. Class sizes are very uneven (Table 1). Yellow leaf curl virus makes up 29.5% of the data and mosaic virus only 2.1%, so a model that ignored the rare classes could still post a respectable accuracy. Figure 1 shows one example per class.

Table 1. Images per class and split. Splits are disjoint by recorded leaf group (Section 2.3). The last column is the share of images in each class that have no entry in the dataset's leaf-grouping file.

Class	Train	Val	Test	Total	Not leaf-grouped
Bacterial spot	1,451	240	436	2,127	0%
Early blight	673	112	215	1,000	0%
Late blight	1,312	219	375	1,906	52%
Leaf mold	668	112	172	952	0%
Septoria leaf spot	1,229	205	337	1,771	44%
Spider mites	1,160	196	320	1,676	0%
Target spot	957	159	288	1,404	100%
Yellow leaf curl virus	3,649	608	1,100	5,357	49%
Mosaic virus	263	44	66	373	100%
Healthy	1,080	179	330	1,589	37%
All classes	**12,442**	**2,074**	**3,639**	**18,155**	**37%**

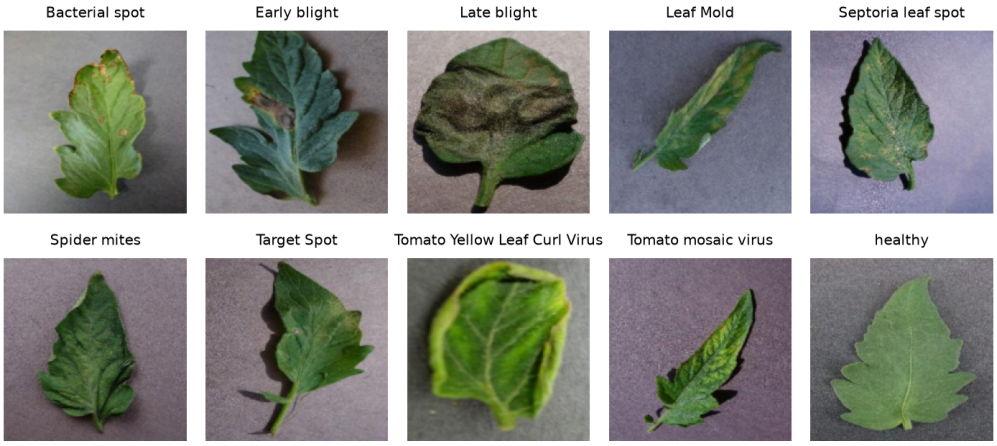


Figure 1. One training image from each of the ten classes. Every photo shows a single leaf on a plain background, which is far easier than the cluttered scenes a grower would photograph in a field.

2.2 Representation

Each image is decoded once, resized from 256×256 to 160×160 pixels with bilinear interpolation, and kept in memory as 8-bit values. Pixels are scaled to [0, 1] and normalized with the ImageNet channel means and standard deviations. The normalization is required for the pretrained network and is applied to all models so that they see identical inputs. We chose 160 pixels so that twelve epochs of every model fit on a laptop. The cost is that very small lesions lose detail; we did not test higher resolutions.

2.3 Split protocol

PlantVillage contains several photographs of the same physical leaf (in the mapped images, groups of four). A random per-image split would put near-identical photos on both sides and inflate the test score. We therefore split by leaf, using the dataset's leaf-grouping file. The **test set** is the official PlantVillage test split restricted to tomato (3,639 images, 1,921 leaf groups). The **validation set** is about 15% of the official training split, drawn with a stratified group split (grouped by leaf, stratified by class, seed 42): 2,074 images in 1,096 groups. The remaining 12,442 images (6,572 groups) form the **training set**. The code checks that no leaf group appears in two splits. Five leaf groups that appeared in both the official training and test lists were removed from training. The split files are committed with the code.

There is one known weakness. The leaf-grouping file covers only about 63% of the tomato images. The other 37% are treated as their own one-image groups, so we cannot guarantee that a different photo of the same leaf is not on the other side of the split. All target spot and mosaic virus images are in this category (Table 1). Section 6.5 measures how much this might matter.

3. Method and Model Design

3.1 Baseline 1: handcrafted features and an SVM

A practitioner without deep learning would first try hand-built features, so we do too. Each image becomes an 88-number vector: 24-bin histograms of hue, saturation and value (72 numbers), a 10-bin histogram of uniform local binary patterns for texture [4] (10 numbers), and the mean and standard deviation of the red, green and blue channels (6 numbers). Features are standardized and fed to an RBF-kernel support vector machine [5] with balanced class weights. The penalty C was chosen from $\{1, 10, 100\}$ on the validation set ($C = 10$). This baseline answers a simple question: how much of the task can be solved by color and texture statistics alone?

3.2 Baseline 2: a small CNN from scratch

The second baseline is a small convolutional network with four blocks of 3×3 convolution, batch normalization, ReLU and 2×2 max pooling (32, 64, 128 and 256 channels), followed by global average pooling, dropout (0.3) and a linear layer. It has 0.39 million parameters. Convolution with shared filters builds in the assumption that useful patterns can appear anywhere in the image (Prince, Chapter 10), which suits lesions that may be anywhere on a leaf. This model shows what a modest, untuned CNN can do with 12,000 training images and no pretraining.

3.3 Main model: ResNet-18 with transfer learning

Our main model is ResNet-18 [6] (11.2 million parameters). Its residual blocks add each block's input to its output, which keeps gradients usable through the stack, and it uses batch normalization [7] (Prince, Chapter 11). It is small enough to fine-tune on a laptop and is a standard choice, which makes it a reasonable "what would a practitioner use" reference. We initialize it from ImageNet weights [8] and replace the final layer with a ten-way linear layer; all layers are fine-tuned. Pretrained features such as edges and textures often transfer to new image tasks [9] (Prince, Chapter 9), and our dataset is small by deep learning standards.

3.4 Training

All networks minimize cross-entropy, which is the maximum-likelihood loss for a categorical output (Prince, Chapter 5). We use Adam [10] with learning rate 10^{-3} and weight decay 10^{-4}, batch size 64, and cosine learning-rate decay [11] to zero over a maximum of 12 epochs. After each epoch we compute macro-F1 on the validation set, keep the best checkpoint, and would stop early after four epochs without improvement (this never triggered). The loss is not class-weighted; we rely on macro-F1 to expose weak classes instead. The SVM and every network use the same 160-pixel images.

3.5 Data augmentation

Augmentation [12] is a regularizer (Prince, Chapter 9): it makes the training set effectively larger by applying transformations that should not change the label. A leaf photographed upside down or at an angle is still the same leaf, and lighting varies between photos. For each training image we apply a random rotation (±20°), zoom (up to 1.25×), shift (±10%), horizontal and vertical flips, and brightness and contrast jitter (±25%), all on the fly and on the GPU. We deliberately do *not* include blur or noise, so that Section 6.4 can test how the models handle corruptions they were not trained for.

3.6 Evaluation measures

Macro-F1 is the primary measure for the reason given in Section 1; accuracy is secondary because it is dominated by the largest classes. We also examine per-class F1, confusion matrices, and which direction the errors go. Since each model is trained once, we quantify uncertainty from the finite test set with a leaf-level bootstrap: whole leaf groups are resampled 1,000 times, so photos of the same leaf stay together, and we report 95% percentile intervals, including for the paired difference between two models on the same resamples.

4. Experimental Setup

Table 2 lists the five runs. The two controlled comparisons change exactly one factor relative to the main model and keep everything else identical: the data split, seed (42), optimizer, schedule, number of epochs and input pipeline. Ablation A (pretrained vs. random initialization) compares runs 3 and 5. Ablation B (augmentation on vs. off) compares runs 4 and 5. Hyperparameters are in Appendix Table A1. Everything ran in PyTorch on a single Apple-silicon laptop using its GPU backend.

Table 2. The five experiments. The best epoch is the one with the highest validation macro-F1 (all deep models ran the full 12 epochs). Training time is wall-clock on a laptop that was sometimes running other jobs, so it is only a rough guide.

#	Model	Init.	Augm.	Params	Best epoch	Train min	Purpose
1	Handcrafted features + SVM	n/a	n/a	–	–	1	non-deep baseline
2	Small CNN	random	on	0.39 M	11	8	deep baseline
3	ResNet-18	random	on	11.2 M	9	14	ablation A
4	ResNet-18	ImageNet	off	11.2 M	9	29	ablation B
5	**ResNet-18 (main)**	ImageNet	on	11.2 M	11	15	main model

Protocol. Every choice (epoch, SVM penalty) is made on the validation set. The test set is scored once per finished model. For the robustness study (Section 6.4) the already-trained models see degraded test images: Gaussian blur with σ of 1, 2 and 4 pixels, additive Gaussian noise with standard deviation 0.05, 0.1 and 0.2 on the [0, 1] pixel scale, and darkening to 50% and 30% brightness. A fixed random seed makes the noise reproducible. The leakage check (Section 6.5) finds, for each test image, its nearest training image by Euclidean distance between 32×32 grayscale thumbnails.

What this design cannot tell us. Each configuration was trained once. The bootstrap intervals capture which test images happened to be drawn, not how much a different random seed would change training. We return to this in Section 6.6.

5. Results

5.1 Overall comparison

Table 3 summarizes test performance. The two pretrained ResNet-18 models are clearly the best, at 0.980 and 0.985 macro-F1. The SVM follows at 0.959, then the randomly initialized ResNet-18 at 0.940 and the small CNN at 0.835. Figure 2 shows how validation macro-F1 evolves. The pretrained networks begin far ahead (0.92 and 0.82 after one epoch, against 0.68 and 0.51 for the from-scratch models) and plateau by about epoch 7, with a dip around epochs 4–5 that is probably the high learning rate meeting the pretrained weights. The from-scratch ResNet needs about nine epochs to reach its plateau. The small CNN is still bouncing between 0.76 and 0.84 at the end, so it is not fully converged; we did not tune it, and its score should be read as "an untuned small CNN", not as a ceiling for small CNNs.

Table 3. Validation and test results (one run each). The bracket is the 95% leaf-level bootstrap interval for test macro-F1; it reflects test-set sampling only.

Model	Val macro-F1	Test accuracy	Test macro-F1	95% interval
Handcrafted features + SVM	0.966	0.965	0.959	[0.949, 0.969]
Small CNN (scratch, aug)	0.844	0.867	0.835	[0.815, 0.852]
ResNet-18 (scratch, aug)	0.955	0.951	0.940	[0.927, 0.952]
ResNet-18 (pretrained, no aug)	0.990	0.989	0.985	[0.980, 0.990]
ResNet-18 (pretrained, aug), main	0.969	0.981	0.980	[0.973, 0.986]

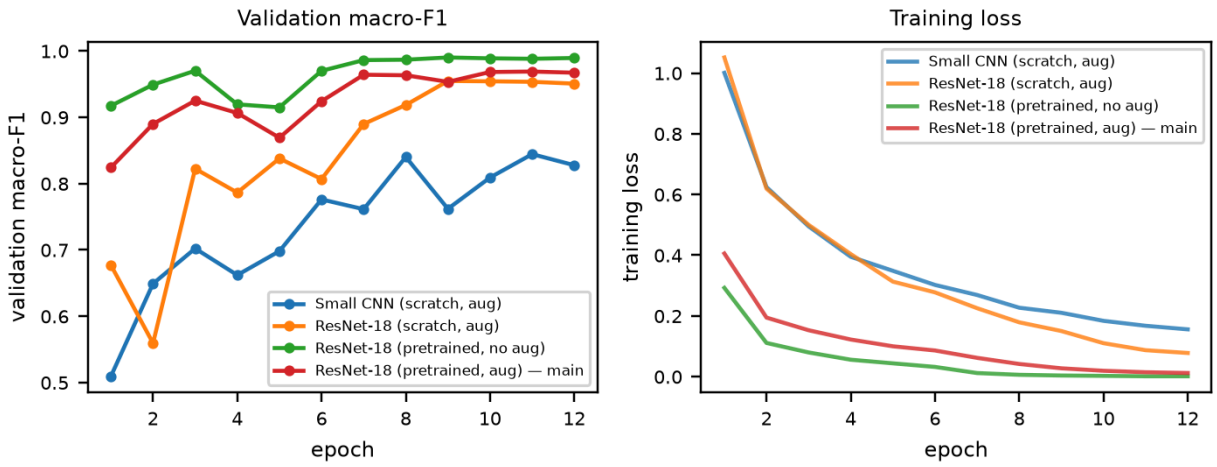


Figure 2. Validation macro-F1 (left) and training loss (right) for the four neural models. Pretraining gives a large head start, and the from-scratch models take most of the 12 epochs to approach it.

5.2 Ablation A: pretraining

With augmentation fixed, ImageNet initialization lifts test macro-F1 from 0.940 to 0.980, a paired difference of +0.039 (95% interval +0.028 to +0.053; Table 4). This is the largest effect in the study and the interval is far from zero. Both runs have the same architecture and parameter count, so the gain comes from the starting point alone. A likely explanation is that 12,000 training images are not many for 11 million parameters, and the pre-

trained filters already encode generic edge and texture detectors that the new task can reuse. We did not test this explanation directly, for example by freezing early layers.

5.3 Ablation B: augmentation

Without augmentation the pretrained model scores slightly higher on clean data: 0.985 against 0.980 on test and 0.990 against 0.969 on validation. The paired test-set difference is −0.006 with an interval of −0.012 to +0.001 (Table 4), so we cannot say the two differ on this test set. The validation gap is larger, and by validation macro-F1 alone we would have selected the unaugmented model. We had designated the augmented model as the main one before seeing results. We keep it as the main model because it is the only one that stays usable under blur and darkening (Section 6.4), but we state plainly that for clean laboratory images alone the unaugmented model would be selected. The training curves point the same way: the unaugmented model reaches 100% training accuracy by epoch 10, yet still generalizes well, which suggests that on this clean, uniform dataset there is little overfitting for augmentation to repair. What augmentation does change is how the model behaves outside the clean test set (Sections 6.2 and 6.4).

Table 4. Paired differences in test macro-F1 on 1,000 leaf-level bootstrap resamples. An interval that excludes zero indicates a difference larger than test-set sampling noise.

Comparison	Difference	95% interval
Pretrained vs. random init. (both augmented): ablation A	+0.039	[+0.028, +0.053]
Augmented vs. unaugmented (both pretrained): ablation B	–0.006	[–0.012, +0.001]
Main ResNet-18 vs. SVM	+0.020	[+0.009, +0.032]
ResNet-18 from scratch vs. SVM	–0.019	[–0.034, –0.005]

5.4 Deep models against the non-deep baseline

The SVM is a strong baseline. It beats the small CNN by 0.12 and the from-scratch ResNet-18 by 0.019 (interval −0.034 to −0.005), and it loses to the pretrained ResNet-18 by 0.020 (interval +0.009 to +0.032). In other words, on this dataset a deep network only clearly pays off when it starts from pretrained weights. This is plausible for lab-style photos, where lesion color and texture separate many classes, and the SVM's features are built from exactly those cues. The per-class view (Table 5) adds detail. The SVM is competitive or better on healthy leaves (0.992 vs. 0.976 for the main model) and target spot (0.953 vs. 0.940), and the small CNN fails on exactly the classes that are rare or visually subtle: healthy (0.673), target spot (0.610), spider mites (0.717) and mosaic virus (0.763).

Table 5. Per-class test F1 for every model. Bold marks the best value in each row. Mosaic virus has only 66 test images, so its value is noisy.

Class	SVM	Small CNN	ResNet scratch	ResNet pre., no aug	ResNet pre., aug (main)
Bacterial spot	0.975	0.939	0.942	**0.994**	0.979
Early blight	0.931	0.861	0.854	0.959	**0.964**
Late blight	0.926	0.902	0.948	**0.991**	0.984
Leaf mold	0.957	0.973	0.958	**0.988**	**0.988**
Septoria leaf spot	0.978	0.935	0.941	0.981	**0.988**
Spider mites	0.936	0.717	0.956	**0.991**	0.986
Target spot	0.953	0.610	0.914	**0.959**	0.940
Yellow leaf curl virus	0.982	0.975	0.974	**1.000**	0.992
Mosaic virus	0.962	0.763	0.930	0.992	**1.000**
Healthy	0.992	0.673	0.988	**0.997**	0.976

Against our criteria from Section 1: the main model beats the SVM by more than sampling noise (Table 4), and the unaugmented model's gap is larger still (+0.026, though we did not compute its interval). Both have F1 of at least 0.94 in every class. The two from-scratch models fail criterion (a), since neither beats the SVM, and both also have classes below 0.90 (criterion b): early blight for the ResNet and several classes for the small CNN.

6. Failure Analysis and Limitations

We analyze the main model (pretrained, augmented), and compare with the unaugmented model where that is informative.

6.1 Where the errors are

The main model makes 68 errors on 3,639 test images (1.9%). Figure 3 shows they are concentrated rather than spread out. The weakest classes by recall are target spot (16 of 288 missed, recall 0.944), early blight (12 of 215, 0.944) and late blight (11 of 375, 0.971); yellow leaf curl virus has the most absolute misses (17) but out of 1,100 images. The two most common confusions are target spot called healthy (12 images) and early blight called target spot (10). The second pair is understandable, since both diseases produce brown spots with ring-like patterns, and it is plausible that early or faint lesions at 160 pixels look like the other disease or like nothing at all. Mosaic virus and healthy leaves are never missed. Precision and recall for every class are in Appendix Table A2; the one weak precision value is for healthy leaves (0.954), which is where the 16 missed diseased leaves of Section 6.2 end up.

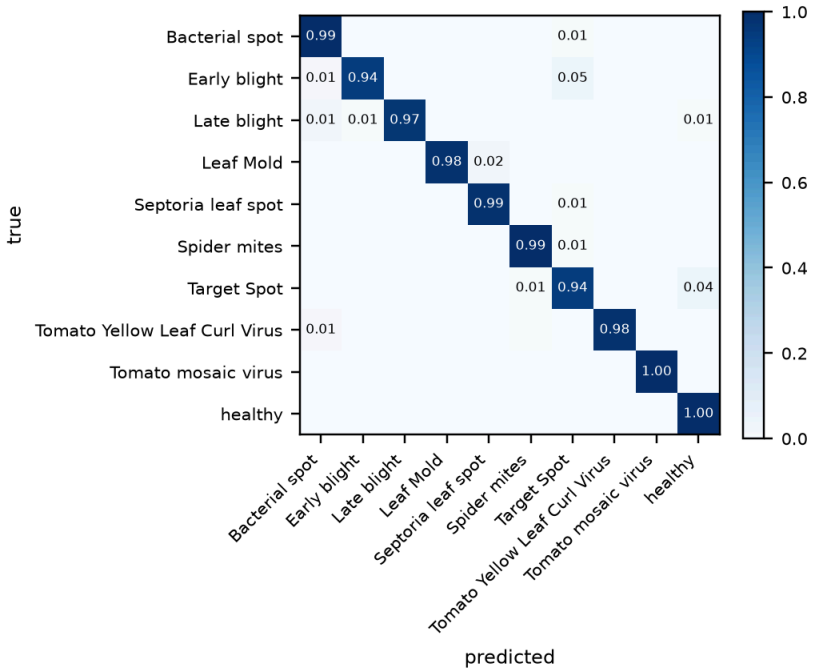


Figure 3. Row-normalized confusion matrix of the main model on the test set (each row sums to 1). Errors cluster around target spot, early blight and late blight.

6.2 The direction of the errors

Not all errors cost the same. A healthy leaf flagged as diseased wastes a treatment; a diseased leaf called healthy can lose a crop. Of the main model's 68 errors, 16 are diseased leaves predicted healthy, and none are healthy leaves predicted diseased (12 of the 16 are target spot). The unaugmented model, with 40 errors, has the oppo-

site profile: no diseased leaf called healthy and two healthy leaves called diseased. Accuracy and macro-F1 hide this difference. If a missed disease is the costly mistake, the unaugmented model is the safer one on clean images, even though the two are statistically tied on macro-F1. We did not investigate why augmentation shifts the errors this way; one possibility is that brightness and contrast jitter weakens the faint color cues that mark early lesions.

6.3 Confident mistakes and what the model looks at

The errors are not marked by low confidence. The main model's average confidence is 0.99 on correct predictions and 0.77 on its errors, but the eight most confident mistakes are made at 0.98–1.00 (all eight are shown in Appendix Figure A1), so a simple confidence threshold would not catch them all. Looking at those eight images, several are leaves where the lesion is small or hard to see, or where the leaf is curled or shadowed. A few are ambiguous enough that we suspect the label itself could be disputed; we did not audit labels.

Figure 4 uses Grad-CAM [13] on the last residual stage to show which regions drive a prediction. For correct and incorrect cases alike, the model's attention sits on the leaf rather than on the background, and in the Septoria example it concentrates on the spotted region. We read this as weak reassurance only. The final feature map is 5×5 at this resolution, so the maps are too coarse to show whether individual lesions are being used, we looked at just eight examples, and attention to the leaf does not rule out a shortcut through subtle background or capture cues. Another study found that a few background pixels alone predict the class in PlantVillage far better than chance [14].

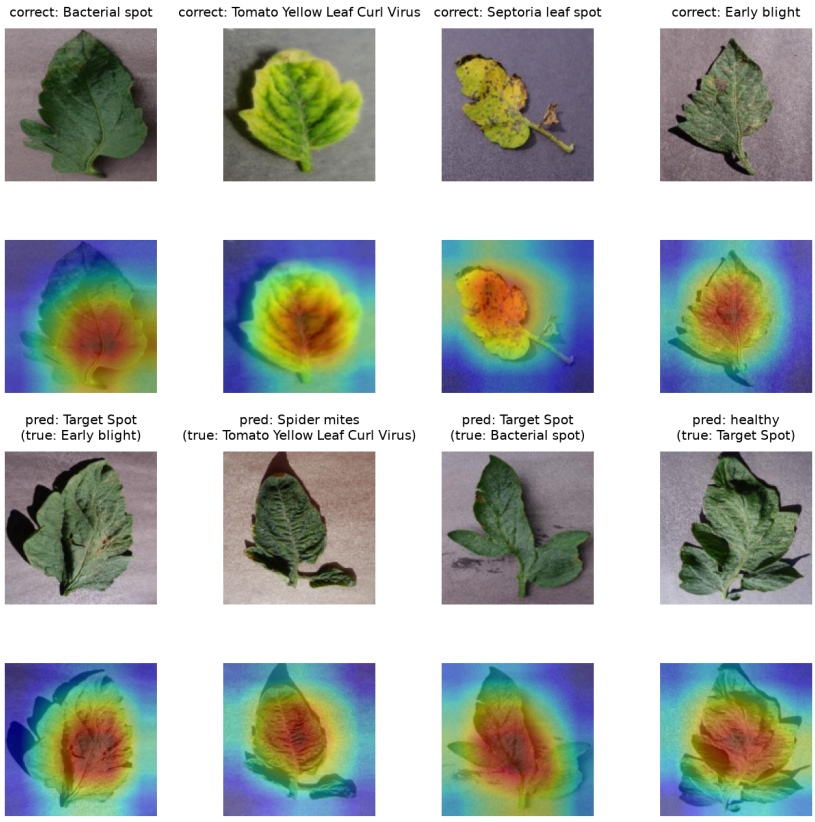


Figure 4. Grad-CAM for the predicted class on four random correct test images (top two rows) and four of the eight most confident errors (bottom two rows; titles give the predicted and true class; all eight are in Appendix Figure A1). In each pair of rows the upper image is the photo and the lower one the overlay; red marks regions that raise the predicted class score.

6.4 Robustness to image degradation

Because real photos are blurry, dark and noisy, we degraded the test images and re-scored three trained models (Table 6, Figure 5). Augmentation matters a great deal here. At blur $\sigma = 1$ the augmented model keeps a macro-F1 of 0.935, but the unaugmented model drops from 0.985 to 0.701. At 50% brightness the numbers are 0.975 and 0.734, and at 30% brightness 0.807 and 0.066. The augmentation recipe included brightness and contrast jitter, which covers darkening directly. It did not include blur, yet it helps there too; one possibility is that the interpolation used when rotating and zooming slightly smooths training images, but we did not test that.

The weak spot is noise. Even Gaussian noise with a standard deviation of 0.05 (visible but modest) cuts the main model to 0.539, and at 0.1 it is 0.082, about what random guessing over ten classes gives. Strong blur ($\sigma \geq 2$) is similarly destructive for all three models. Augmentation protects against the shifts it simulates and little else. Since the models appear to depend on fine texture, a grower's slightly blurry or noisy phone photo is a realistic way for them to fail.

Table 6. Test macro-F1 when the test images are degraded. The models were not retrained or adapted. Augmentation (main) covered brightness and contrast but not blur or noise.

Condition	ResNet-18 pre., aug (main)	ResNet-18 pre., no aug	Small CNN
Clean	0.980	0.985	0.835
Blur σ=1	0.935	0.701	0.652
Blur σ=2	0.398	0.280	0.371
Blur σ=4	0.164	0.122	0.263
Noise σ=0.05	0.539	0.524	0.217
Noise σ=0.1	0.082	0.018	0.039
Noise σ=0.2	0.032	0.017	0.019
Brightness ×0.5	0.975	0.734	0.617
Brightness ×0.3	0.807	0.066	0.119

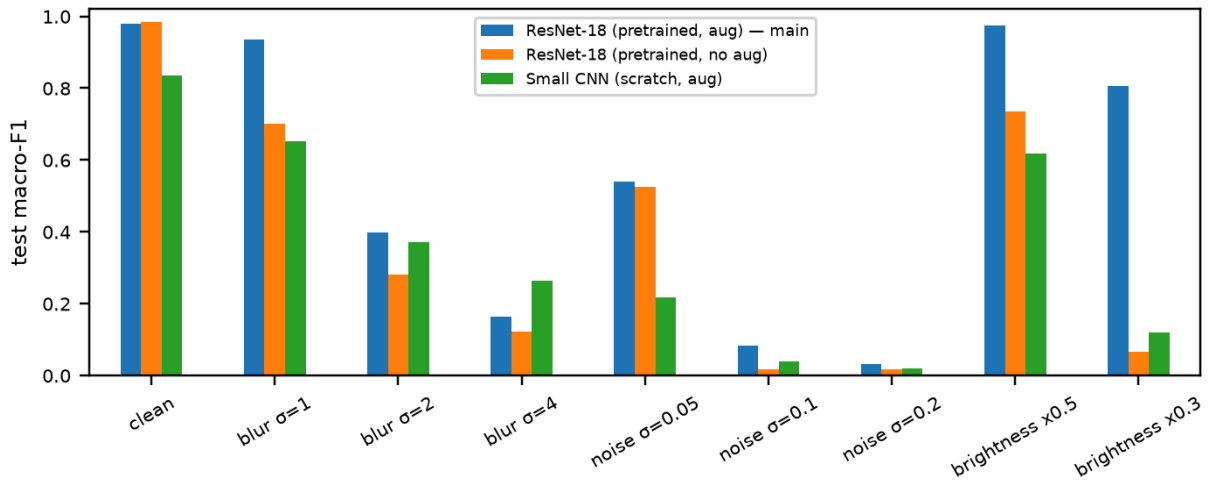


Figure 5. The data of Table 6 as a bar chart. Augmentation (blue) is far more robust than no augmentation (orange) to blur and darkening, and no model survives noise.

6.5 Possible leakage through the incomplete leaf grouping

As noted in Section 2.3, 1,348 of the 3,639 test images (37%) have no leaf-group entry. Figure 6 compares how close each test image is to its nearest training image for the grouped and ungrouped images. The ungrouped im-

ages are closer on average (median distance 2.50 vs. 3.17) and have an extra cluster of very close matches (distance below about 1.8) that the grouped images lack. That is what near-duplicate leakage would look like. On the other hand, the main model is not more accurate on the ungrouped images (97.6% vs. 98.5% for grouped), which argues against the leakage having inflated the scores much. We cannot treat this as settled. The two groups also differ in class mix, and the ungrouped set contains every target spot and mosaic virus image, so the near-perfect mosaic score (F1 1.000 on 66 images) is the number we trust least.

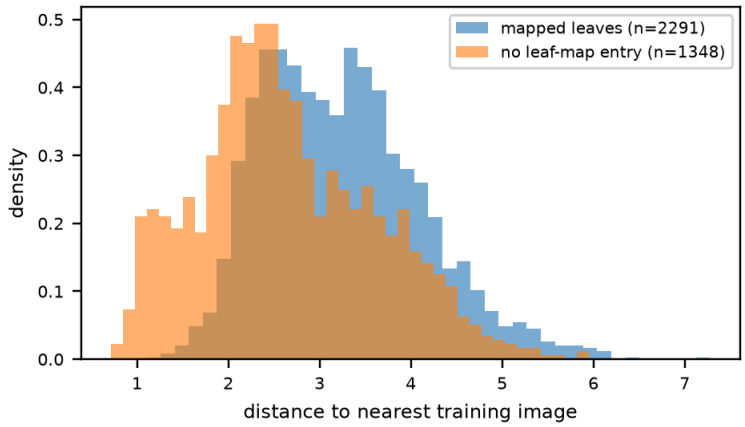


Figure 6. Distance from each test image to its nearest training image (32×32 grayscale thumbnails), for test images whose leaf is covered by the leaf-group file (blue) and those that are not (orange). The ungrouped images include a cluster of unusually close matches.

6.6 Limitations

- **Laboratory images only.** Every photo is a single leaf on a plain background. We did not test on field photographs, so the scores here almost certainly overstate real-world accuracy. How large that shortfall is remains the open question.
- **One training run per model.** Seed-to-seed variation is unmeasured. Differences of a few tenths of a point, including the augmentation comparison on clean data, should not be over-read. The pretraining effect (+0.039) and the gaps to the SVM are large relative to the test-set intervals, but could still partly reflect a lucky or unlucky run.
- **Incomplete leaf grouping.** Section 6.5 shows we cannot exclude some leakage for the 37% of images with no grouping information.
- **Untuned baselines and a short schedule.** The small CNN is not converged, the learning rate was not tuned for fine-tuning, and every model got 12 epochs at 160 pixels. A better-tuned small CNN would narrow the gap.
- **Narrow scope.** One crop and ten classes; no unknown or "other" class, so the model must pick one of the ten labels for anything it sees. Class imbalance leaves the rare classes with few test images.

7. Ethics, Limitations and Responsible Use

Over-trust and the cost of mistakes. A classifier like this would be used by people deciding whether to spray, remove plants, or call an expert. Our analysis shows two properties that make over-trust risky: wrong answers are often given with very high confidence (Section 6.3), and the model that looks best on average can have the more dangerous error profile, since the augmented model never raises a false alarm but misses diseased leaves (Section 6.2). Any real tool should report that it is uncertain when it is, present results as a suggestion for expert

confirmation, and track the recall of each disease and not only overall accuracy. A false alarm also has a cost, in unnecessary pesticide use and its environmental impact, so neither direction of error is harmless.

Dataset bias and who is left out. The training photos come from controlled conditions, one crop and a limited set of disease appearances. The model could perform worse on other tomato varieties, growth stages, regions, lighting and cheap phone cameras. Because the people with the least access to experts are also likely to work under the conditions furthest from the laboratory, a tool that works well only on clean photos would serve them worst. The earlier report that such models can fall sharply on photos from different conditions [1] and our own robustness results (Section 6.4) both point the same way. The data may also carry unintended shortcuts: classes have been shown to be partly predictable from a handful of background pixels [14], and such cues would not exist in the field. Field-image benchmarks such as PlantDoc [15] are the natural next test; we did not run one, and we therefore make no claim about field accuracy.

Scope and misuse. The model knows ten tomato classes. A different crop, a nutrient deficiency, pesticide damage or a plant that is simply stressed will still be forced into one of the ten classes, probably with high confidence. Deployment would need an out-of-scope or abstain option. The risk of deliberate misuse is low for this task, but a tool wrongly presented as a diagnosis could mislead growers or be used to dismiss legitimate advice.

Privacy, licensing and compute. Leaf photos carry little personal information, but photos taken in the field can include location data, people or property; any collection effort should strip such metadata and obtain consent. The data is released under CC-BY-SA-3.0, so we cite its creators and do not redistribute the images. All training took roughly an hour of laptop time in total (about 1.5 hours including the analysis), a small footprint, though repeated tuning and larger models would multiply it and that cost should be reported.

Recommended use. Treat the system as a screening aid that tells a grower which plants to look at more closely, never as a final diagnosis, and validate it on photos from the intended users and conditions before anyone relies on it.

8. Conclusion

On tomato leaves from PlantVillage, a pretrained ResNet-18 identifies ten classes with a test macro-F1 of 0.980 (0.985 without augmentation), clearly ahead of a handcrafted-feature SVM at 0.959, which itself beats two networks trained from scratch. The most useful finding for practice is that the starting point (ImageNet pretraining) mattered more than anything else we varied, with a gain of about 0.04 macro-F1. Augmentation did not change clean accuracy in a measurable way but kept the model working under blur and darkening, and it changed which mistakes the model makes, moving errors toward missed disease. Every model degraded sharply under image noise, errors are made with high confidence, and our leaf-level split cannot be fully verified for 37% of the images, so the scores should be taken as an upper bound on field performance.

Next steps. The most valuable follow-ups are: evaluating on real field photographs (for example PlantDoc); repeating each run over several seeds to put intervals on the training variation; adding noise and blur to training and measuring the effect; trying a higher input resolution for small lesions; and adding calibrated confidence with an abstain option. A perceptual-hash grouping of the ungrouped images would also tighten the leakage guarantee.

References

1. S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” *Frontiers in Plant Science*, vol. 7, Art. 1419, 2016.
2. S. J. D. Prince, *Understanding Deep Learning*. Cambridge, MA, USA: MIT Press, 2023.

3. D. P. Hughes and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” arXiv:1511.08060, 2015.
4. T. Ojala, M. Pietikäinen, and T. Mäenpää, “Multiresolution gray-scale and rotation invariant texture classification with local binary patterns,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 971–987, 2002.
5. C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
6. K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
7. S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456.
8. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE CVPR*, 2009, pp. 248–255.
9. J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How transferable are features in deep neural networks?” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014, pp. 3320–3328.
10. D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.
11. I. Loshchilov and F. Hutter, “SGDR: Stochastic gradient descent with warm restarts,” in *Proc. ICLR*, 2017.
12. C. Shorten and T. M. Khoshgoftaar, “A survey on image data augmentation for deep learning,” *Journal of Big Data*, vol. 6, Art. 60, 2019.
13. R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 618–626.
14. M. A. Noyan, “Uncovering bias in the PlantVillage dataset,” arXiv:2206.04374, 2022.
15. D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat, and N. Batra, “PlantDoc: A dataset for visual plant disease detection,” in *Proc. 7th ACM IKDD CoDS and 25th COMAD*, 2020, pp. 249–253.

Appendix A. Reproducibility Details

All code is in the notebook `plant_leaf_disease.ipynb` in the repository named on the title page. Running it from top to bottom downloads the data (about 2.2 GB), builds the split (committed in `data/splits/`), trains all models, and regenerates every table and figure in this report; finished runs are cached under `results/`. The pinned dependencies are in `requirements.txt` (and `environment.yml` for conda), the settings in Table A1 are read from `config.yaml`, and the weights of the main model are saved in `results/resnet18_pretrained_aug.pt`. The weights of the other three networks are saved alongside it in `results/`. The full run took about 1.5 hours on an Apple-silicon laptop.

Table A1. Hyperparameters and settings shared by all neural models.

Setting	Value
Input size / normalization	160×160, ImageNet mean and standard deviation
Loss	Cross-entropy, no class weights, no label smoothing
Optimizer	Adam, learning rate 1e-3, weight decay 1e-4
Schedule	Cosine decay to zero over 12 epochs
Batch size / epochs / early stopping	64 / 12 / patience 4 on validation macro-F1 (never triggered)
Dropout	0.3 in the small CNN's head; none in ResNet-18
Augmentation (when on)	Rotation ±20°, zoom up to 1.25×, shift ±10%, horizontal and vertical flip, brightness and contrast ±25%
Seed	42 (data split, initialization, shuffling, noise)
SVM	RBF kernel, gamma = "scale", balanced class weights, C = 10 chosen from {1, 10, 100}
Bootstrap	1,000 resamples of test leaf groups, 95% percentile intervals

Table A2. Precision and recall of the main model on the test set (support = number of test images).

Class	Precision	Recall	Support
Bacterial spot	0.964	0.993	436
Early blight	0.985	0.944	215
Late blight	0.997	0.971	375
Leaf mold	0.994	0.983	172
Septoria leaf spot	0.988	0.988	337
Spider mites	0.978	0.994	320
Target spot	0.935	0.944	288
Yellow leaf curl virus	0.999	0.985	1,100
Mosaic virus	1.000	1.000	66
Healthy	0.954	1.000	330

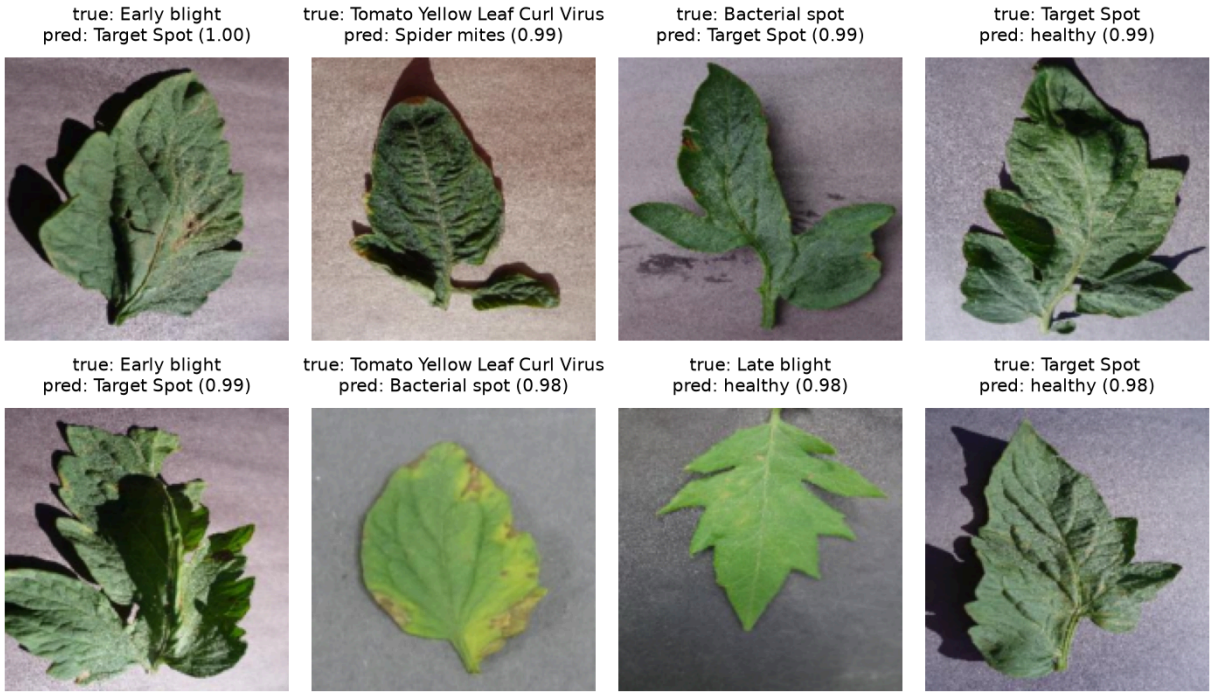


Figure A1. The eight most confident test errors of the main model, with true label, predicted label and confidence. Faint lesions, curled leaves and heavy shadows are common. Four of these also appear in Figure 4.